Basic SQL Queries and Aggregate Functions

Filtering, sorting, grouping, HAVING, and five aggregate functions

Prepared by: Vigneshvar K K

Project: Library Management System. **Output convention:** result tables below are reproducible expected outputs from the fixed rows in the shared SQL script. They are not screenshots of MySQL execution.

1. Filtering with WHERE

This query returns catalog books priced at 600.00 or more. ORDER BY makes the output easy to check.

```
SELECT book_id, title, list_price
FROM books
WHERE list_price >= 600.00
ORDER BY list_price DESC, title;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

book_id	title	list_price
3	Python in Practice	720.00
7	Learning Web Design	680.00
1	Relational Database Basics	650.00

2. Sorting with ORDER BY

```
SELECT title, list_price
FROM books
ORDER BY list_price DESC, title;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

title	list_price
Python in Practice	720.00
Learning Web Design	680.00
Relational Database Basics	650.00
Data Structures Made Clear	590.00
Mystery at Lake Road	550.00
SQL for Beginners	520.00
The River Beyond	480.00
A Short History of Bengal	430.00

The query sorts by list price, then title. For a clearer ascending/descending report, the order can be changed explicitly; this example keeps the price sort descending.

3. Aggregate functions

COUNT, SUM, AVG, MIN, and MAX summarize the full catalog.

```
SELECT COUNT(*) AS book_count,
       SUM(list_price) AS catalog_value,
       AVG(list_price) AS average_price,
       MIN(list_price) AS lowest_price,
       MAX(list_price) AS highest_price
FROM books;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

book_count	catalog_value	average_price	lowest_price	highest_price
8	4620.00	577.500000	430.00	720.00

4. GROUP BY and HAVING

The first query groups books by category. HAVING then keeps only categories with at least two books.

```
SELECT c.category_name,
       COUNT(b.book_id) AS book_count,
       SUM(b.list_price) AS category_value,
       AVG(b.list_price) AS average_price
FROM categories AS c
JOIN books AS b ON b.category_id = c.category_id
GROUP BY c.category_id, c.category_name
ORDER BY c.category_name;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

category_name	book_count	category_value	average_price
Database	2	1170.00	585.000000
Fiction	2	1030.00	515.000000
History	1	430.00	430.000000
Programming	3	1990.00	663.333333

The HAVING result excludes History because it contains one book.

```
SELECT c.category_name, COUNT(b.book_id) AS book_count
FROM categories AS c
JOIN books AS b ON b.category_id = c.category_id
GROUP BY c.category_id, c.category_name
HAVING COUNT(b.book_id) >= 2
ORDER BY c.category_name;
```

Expected output from the deterministic sample rows in Library_Management_System_MySQL_8.sql:

category_name	book_count
Database	2
Fiction	2
Programming	3

Screenshot steps

Run each query separately in MySQL Workbench and capture the statement with its Result Grid. Use the fixed catalog seed data from Assignment 7 so the numbers match the expected output tables above.